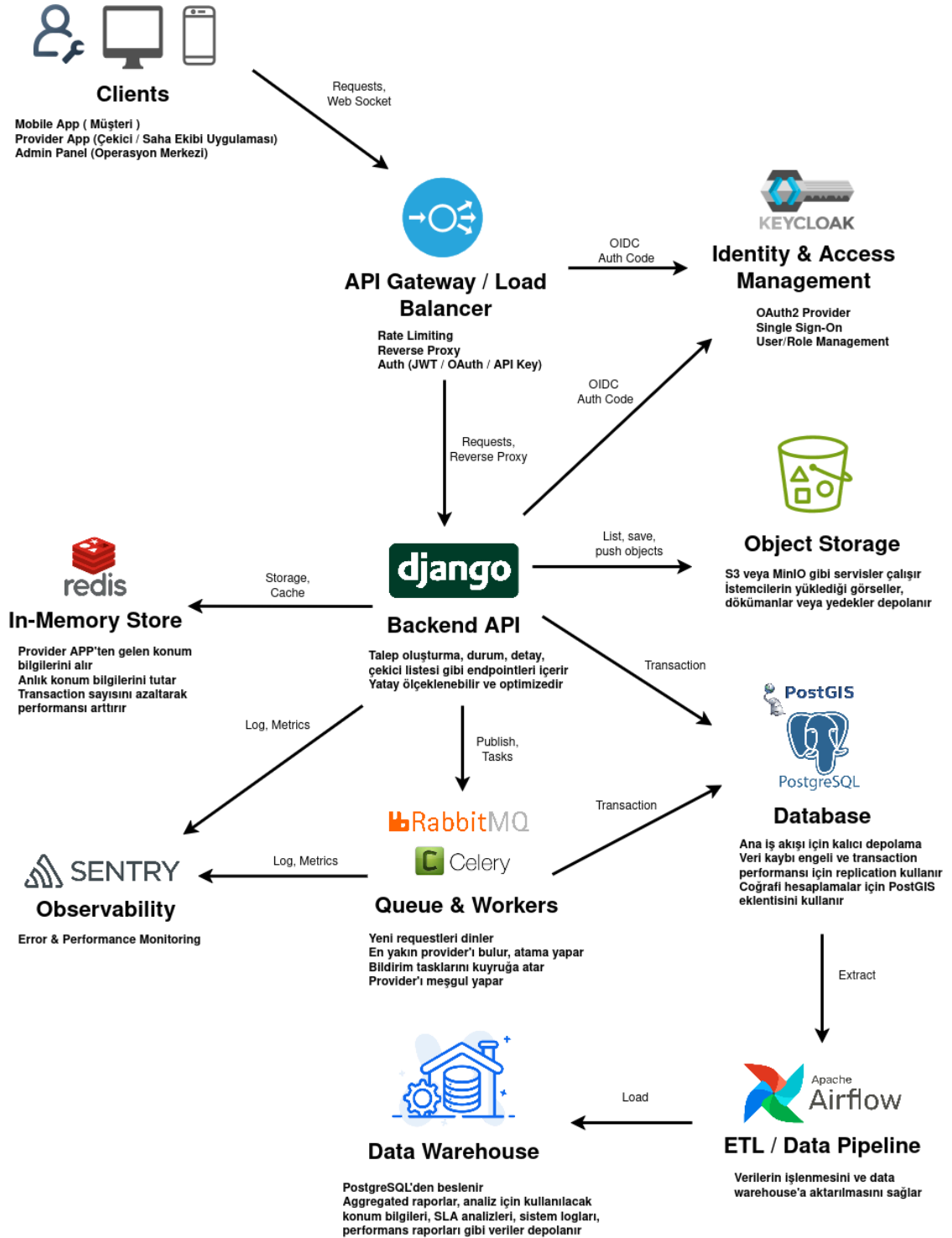


ASSISTANCE UYGULAMASI (Backend Case Study)

Belge, sistem tasarımı (mimari kurgulama), kodlama açıklamaları ve code review olmak üzere üç bölümden oluşmaktadır. Task açıklamalarını içerir.

1) Sistem Tasarımı



a) Talep Karşılama ve Kuyruklama

Soru:

Bayram gibi yoğun dönemlerde on binlerce talep aynı anda gelebilir; API'yi ve veritabanını kilitlemeden bu yükü nasıl karşılarız?

Tasarım Kararı:

Stateless API + Yatay Ölçekleme

Gelen istekler önce bir API gateway / load balancer üzerinden geçerek birden fazla backend API (Django) instance'larına dağıtılıyor. API katmanı session tutmadığı için gerektiğinde yeni instance ekleyerek yatay ölçeklemek mümkün.

İşlemeyi Kuyruğa Bırakma

Yeni bir yardım talebi geldiğinde API sadece kaydı alır ve işlemi kuyruğa bırakır. Müşteriye hemen status bilgisi döner.

Cache Mekanizması Kullanımı

Redis ile yoğun olarak işlenen veriler memory'de tutulur ve veritabanı sorguları azaltılarak yüksek performans sağlanır.

Kuyruklar

Yoğunluk anlarında sistem anlık olarak tüm talepleri işleyemese bile talep alınır ve güvenli bir şekilde kuyrukta bekletilir. Böylelikle yüksek yoğunlukta dahil talep sıraya alınır ve sistem kararlılığı korunur.

Worker Sayısı

Yoğunluğa göre worker sayısı artırılarak işleme kapasitesi yükseltilebilir.

b) Entegrasyon ve Dayanıklılık

Soru:

Sigorta şirketlerinin servisleri yanıt vermediğinde veya yavaşladığında sisteminizin geri kalanı (mobil uygulama, admin paneli) nasıl etkilenmez?

Tasarım Kararı:

Entegrasyonun API'den Tamamen Ayrılması

Sigorta şirketine bildirim, Backend API içerisinde senkron yapılmıyor. Talep işleme yavaşladığında yalnızca worker kapasitesi dolmuş olur, API tarafı talep almaya ve provider atamaya devam eder. Worker sayısı ölçeklendirildiğinde, talep işleme süresi azalır.

Retry + Exponential Backoff + Circuit Breaker mekanizmaları

Celery tarafında belirli sayıda retry ile denenir ve hatalarda bekleme üstel olarak artıyor. Sürekli hata veriyorsa circuit breaker devreye giriyor. Böylelikle sistem boğulmuyor.

Veri Tutarlılığı

Dış servis tamamen kapalı olsa bile operasyonel akış devam eder.

c) Gerçek Zamanlı İletişim

Soru:

Sahadaki çekicinin konumu ve durumu değiştiğinde, müşteriye ve operasyon merkezine bu bilgiyi en verimli şekilde nasıl iletirsiniz?

Tasarım Kararı:

Bağımsız Tracking/Realtime Servisi

Çekici uygulaması, konum ve durum güncellemelerini, konum bilgilerini Redis'te tutan ayrı bir backend servisine gönderiyor. Bu sayede yüksek frekanslı konum güncellemeleri, PostgreSQL'i zorlamıyor.

Durum Değişikliklerinin Olay Bazlı Yönetimi

Çekicinin durum bilgisi, Backend servisine event olarak iletiliyor. Bu eventler hem veritabanında işleniyor hem de message broker üzerinden diğer servislere raporlama amacıyla paylaşılabiliyor.

d) Veri Saklama ve Raporlama

Soru:

Operasyonel veriler ile raporlama/analitik verilerini nasıl yönetirsiniz?

Çözüm:

Operasyonel Veri İçin PostgreSQL

Canlı operasyon için AsistanceRequest, Provider, ServiceAssignment gibi tablolar PostgreSQL içinde tutuluyor. Bu veritabanı sadece online işlem trafiğine göre optimize ediliyor.

Analitik veriler için DWH ve Object Storage

Ağır sorgular ve raporlar için PostgreSQL'den düzenli olarak ETL ile Data Warehouse'a veriler taşınıyor. Yüksek hacimli event verileri (racking veya log gibi) object storage'a (MinIO veya S3 gibi) partition'lı dosyalar olarak yazılabiliyor. Ayrıca kullanıcılardan gelen görsel, metin dosyası gibi formatlar da object storage'a kaydoluyor.

Verinin Yaşam Döngüsü ve Arşivleme

Eski kayıtlar veya arşiv verileri, DWH alanında saklanıp gerektiğinde okunabiliyor. Bu sayede hem maliyet hem de operasyonel performans kontrol altında tutuluyor.

2) Kodlama

Düzenlenmiş iş mantığı kodları github reposuna yüklenmiştir. Kodlama görevi açıklamaları aşağıdaki gibidir:

a) En Yakın Provider Bulma

Metod: find_nearest_available_provider(lat, lon)

Önce sadece müsait provider'ları seçiyorum

```
qs = Provider.objects.filter(is_available=True)
```

Performans için tüm provider'ları çekmek yerine basit bir bounding box uyguluyorum. LAT_DELTA ve LON_DELTA sabitleri ile kullanıcının etrafında yaklaşık +0.2 derece (kabaca +-20km civarı) bir kutu tanımlıyorum. Böylece coğrafi olarak çok uzak olan provider'ları daha baştan eleniyor.

```
qs = qs.filter(
    lat__gte=lat - LAT_DELTA,
    lat__lte=lat + LAT_DELTA,
    lon__gte=lon - LON_DELTA,
    lon__lte=lon + LON_DELTA,
)
```

Eğer for_update=True parametresi verilmişse; bu fonksiyonun bir transaction.atomic bloğu içinde çağırıldığını varsayıyorum. Queryset'e select_for_update() ekleyerek provider satırlarını row-level lock ile kilitliyorum. Böylece aynı anda başka bir transaction bu provider'ı atamaya çalışırsa, veri tutarlılığı korunmuş oluyor.

Sonuç setini bir limit ile sınırlandırıyorum. Bunun sonucunda hem bellek hem de hesaplama yükünü kontrol altında tutuyorum:

```
providers = list(qs[:limit])
```

Bounding box içerisinde hiç provider yoksa, fallback yaparak global olarak uygun provider'lar arasında yine limit kadar kayıt alıyorum. Eğer hala provider yoksa ValueError fırlatıyorum.

En sonunda Python tarafında Haversine formülü ile mesafe hesaplıyorum.

Dipnot: Gerçek hayat senaryosunda daha verimli işleme yöntemleri için PostGIS eklentisine başvurularak bu tür işlemler veritabanı katmanında gerçekleştirilebilir.

b) Race Condition Fix

Metod: assign_provider_atomic(request_id, provider_id=None)

Aynı anda gelen iki talebin aynı provider'ı kapmaya çalışmasını aşağıdaki yapı ile engelledim:

Öncelikle metodun tamamını transaction.atomic() bloğu içerisinde çalıştırıyorum. Böylelikle provider'ın is_available flagı, request'in statüsü, ServiceAssignment kaydının oluşturulması ya hep ya hiç kuralıyla uygulanıyor.

İlk olarak AssistanceRequest satırını select_for_update() ile kilitliyorum:

```
req = AssistanceRequest.objects.select_for_update().get(
    id=request_id
)
```

Böylece aynı request için assign_provider_atomic iki kere çağırılrsa bile ikinci çağrı ilk transaction bitene kadar bekliyor.

Eğer id verilmişse o provider'ı is_available=True filtresi ile alıyorum. Id verilmemiş ise find_nearest_available_provider(..., for_update=True) metodu çağırarak provider satırını select_for_update ile kilitliyorum.

Böylece hem request hem provider satırları bu transaction süresince kilitli oluyor ve aynı provider'ın iki farklı talebe aynı anda ataması engelleniyor.

En kritik noktalardan birisi ise bildirim sadece veritabanı işlemi kesinlikle başarılı olduktan sonra gönderilmesidir. Bunun için transaction.on_commit() callback'ini kullanıyorum:

```
transaction.on_commit(
    lambda: notify_insurance_company_task.delay(req.id)
)
```

Bu sayede transaction başarılı olursa, Celery task'ı kuyruğa atılıyor. Rollback olursa, callback hiç çalışmıyor.

Sonuç:

Race condition'lar veritabanı seviyesinde engelleniyor.

Atama süreci tek transaction'da tutuluyor

Dış sistemle entegrasyon, sadece kesinleşmiş durumda tetikleniyor.

c) Celery Retry Mekanizması

Celery taskı içerisinde exception oluşursa aşağıdaki kod bloğu devreye giriyor:

```
retries = self.request.retries
countdown = 2 ** retries
logger.warning(...)

try:
    raise self.retry(exc=exc, countdown=countdown)
except self.MaxRetriesExceededError:
    logger.error(...)
    raise
```

Burada hata oluştuğunda hemen peş peşe tekrar denemek yerine her seferinde bekleme süresini arttırarak dış servise yük bindirmeyen bir retry mekanizması elde ettim.

Maksimum retry sayısına ulaşıldığında Celery, self.retry içinde MaxRetriesExceededError fırlatıyor. Bu exception'ı yakalayıp hatanın logunu atıyorum. Böylece hem başarısız taskları kaydediyorum hem de sistem bozulursa sonsuza kadar retry döngüsüne girmiyor.

d) Complete & Cancel İşlemleri

İş kuralında sadece "DISPATCHED" olan talepler tamamlanabilir. Provider'ı tekrar müsait hale getirilebilir ve yeni talep alınabilir.

İşlem transaction.atomic() bloğu içerisinde ve select_for_update() ile kitlendiğinden aynı talep üzerinde **aynı anda hem cancel hem de complete gibi çalışan işlemler yapılamıyor.**

3) Code Review

Tespit ettiğim en kritik 3 hata açıklamaları ile aşağıdaki gibidir:

a) Tüm Veriyi Memory'e Çekmek

Aşağıdaki kod satırlarında ihtiyaç duyulan obje sayısını hesaplamak için tüm assistance request objeleri, veritabanından çekilerek memory'de indexleniyor. Bunun sonucunda veri büyüdükçe bellek şişer ve performans düşer.

```
all_requests = AssistanceRequest.objects.all()
total_count = len(all_requests)
```

Doğru çözüm aşağıdaki gibi; toplam sayının veritabanında hesaplanmasıdır.

```
total_count = AssistanceRequest.objects.count()
```

b) Filtrelemenin Servis Katmanında Olması + Timezone Sorunu

Aşağıdaki kod satırları tüm provider objelerini memory'e çekiyor ve filtrelemeyi servis katmanında gerçekleştiriyor. Provider sayısı büyüdüğünde ciddi performans düşmelerine neden olur. Servisin çalıştığı ortama bağlı olarak yanıt süreleri artar, istekler time out değerlerinin üstüne çıkar.

```
providers = Provider.objects.all()
for p in providers:
    if p.is_active and p.last_ping > datetime.now() -
timedelta(minutes=5):
        active_providers.append(p)
```

Doğru çözüm aşağıdaki gibi; filtreleme işleminin veritabanı katmanında gerçekleştirilmesi ve Django varsayılanı olan timezone-aware kullanılmasıdır:

```
cutoff = timezone.now() - timedelta(minutes=5)

active_count = Provider.objects.filter(
    is_active=True,
    last_ping__gt=cutoff,
).count()
```


c) Log Sorgusunda Sıralama Olmaması + Bilgi Sızıntısı

Aşağıdaki kod satırlarında **[5]** öncesi **order_by** olmadığı için veritabanı rastgele 5 satırı dönebilir. Ayrıca hata loglarını direkt yanıtta eklemek hassas veri, internal sistem adı, stack trace gibi bilgi sızıntılarına sebep olabilir.

```
logs = Log.objects.filter(level='ERROR')[5]
log_messages = [l.message for l in logs]
return {
    "total": total_count,
    "active": len(active_providers),
    "logs": log_messages
}
```

Doğru çözüm aşağıdaki gibi olmalıdır. Ayrıca hata logları ayıklanabilir veya Sentry, Kibana, Grafana gibi araçlara yönlendirilebilir.

```
logs = (
    Log.objects.filter(level='ERROR')
    .order_by('-created_at')[5]
    .values_list('short_message', flat=True)
)
```